ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Retrieve and manipulate data with SQL
- WHERE, ORDER BY, GROUP BY and HAVING
- Aggregate functions COUNT(), SUM(), AVG(), MIN(), MAX()
- SQL commands with their outputs and brief explanations

1. Introduction

This assignment demonstrates the fundamental SQL clauses on the Airline Reservation System database loaded in Assignment 7. Every query below was executed and the output shown is the actual result (SQLite 3.45; the SQL is standard and runs unchanged on MySQL and PostgreSQL). Fares are in INR. Queries are numbered Q1 to Q24.

2. Retrieving and Filtering Data (SELECT, WHERE)

Q1 List of airports

SQL

```sql
SELECT airport_code, city, country FROM airport;
```

OUTPUT

```
+--------------+-----------+----------------------+
| airport_code | city      | country              |
+--------------+-----------+----------------------+
| BLR          | Bengaluru | India                |
| DEL          | New Delhi | India                |
| BOM          | Mumbai    | India                |
| MAA          | Chennai   | India                |
| HYD          | Hyderabad | India                |
| CCU          | Kolkata   | India                |
| COK          | Kochi     | India                |
| GOI          | Goa       | India                |
| DXB          | Dubai     | United Arab Emirates |
| SIN          | Singapore | Singapore            |
| LHR          | London    | United Kingdom       |
+--------------+-----------+----------------------+
11 rows in set
```

Explanation: SELECT with named columns returns only the columns needed instead of the whole row.

Q2 Flights departing from Bengaluru that are not cancelled

```sql
SELECT flight_number, destination_code, departure_time, base_fare, status
FROM flight
WHERE origin_code = 'BLR' AND status <> 'Cancelled';
```

```
OUTPUT
+---------------+------------------+---------------------+-----------+-----------+
| flight_number | destination_code | departure_time      | base_fare | status    |
+---------------+------------------+---------------------+-----------+-----------+
| AI-807        | BOM              | 2026-10-21 07:30:00 |      4100 | Completed |
| AI-807        | BOM              | 2026-11-02 07:30:00 |      4300 | Scheduled |
| 6E-319        | COK              | 2026-11-08 16:30:00 |      3200 | Scheduled |
| 6E-501        | DEL              | 2026-10-20 06:00:00 |      5000 | Completed |
| 6E-501        | DEL              | 2026-11-02 06:00:00 |      5200 | Scheduled |
| EK-565        | DXB              | 2026-11-04 03:50:00 |     21500 | Scheduled |
| QP-1305       | HYD              | 2026-11-03 08:00:00 |      2800 | Scheduled |
| 6E-231        | MAA              | 2026-11-03 18:30:00 |      2500 | Scheduled |
+---------------+------------------+---------------------+-----------+-----------+
8 rows in set
```

Explanation: WHERE keeps only rows satisfying both conditions combined with AND; <> means 'not equal'.

Q3 Flights in a date range within a fare band

```sql
SELECT flight_number, origin_code, destination_code, departure_time, base_fare
FROM flight
WHERE departure_time BETWEEN '2026-11-02 00:00:00' AND '2026-11-04 23:59:59'
  AND base_fare BETWEEN 3000 AND 6000;
```

```
OUTPUT
+---------------+-------------+------------------+---------------------+-----------+
| flight_number | origin_code | destination_code | departure_time      | base_fare |
+---------------+-------------+------------------+---------------------+-----------+
| 6E-501        | BLR         | DEL              | 2026-11-02 06:00:00 |      5200 |
| 6E-502        | DEL         | BLR              | 2026-11-02 10:00:00 |      5100 |
| AI-807        | BLR         | BOM              | 2026-11-02 07:30:00 |      4300 |
| AI-808        | BOM         | BLR              | 2026-11-02 11:00:00 |      4200 |
| SG-612        | DEL         | BOM              | 2026-11-03 09:15:00 |      3900 |
| AI-505        | DEL         | CCU              | 2026-11-04 14:00:00 |      4600 |
+---------------+-------------+------------------+---------------------+-----------+
6 rows in set
```

Explanation: BETWEEN is inclusive on both ends. Two BETWEEN tests are combined: departures from 2 to 4 November with a base fare of 3000 to 6000.

Q4 Pattern search: passengers whose surname ends with 'Reddy' or e-mail starts with 'd'

```sql
SELECT passenger_id, first_name, last_name, email
FROM passenger
WHERE last_name LIKE '%Reddy' OR email LIKE 'd%';
```

```
OUTPUT
+--------------+------------+-----------+---------------------------+
| passenger_id | first_name | last_name | email                     |
+--------------+------------+-----------+---------------------------+
|            6 | Sneha      | Reddy     | sneha.reddy@example.com   |
|            7 | Arjun      | Reddy     | arjun.reddy@example.com   |
|            8 | Kavya      | Reddy     | kavya.reddy@example.com   |
|           10 | Divya      | Krishnan  | divya.krishnan@example.com |
|           19 | David      | Thomas    | david.thomas@example.com  |
+--------------+------------+-----------+---------------------------+
5 rows in set
```

Explanation: LIKE with % matches any number of characters. OR returns rows meeting either condition.

Q5 IN list: flights operated to Gulf and Asian destinations

```sql
SELECT flight_number, origin_code, destination_code, base_fare
FROM flight
WHERE destination_code IN ('DXB', 'SIN', 'LHR');
```

```
OUTPUT
+---------------+-------------+------------------+-----------+
| flight_number | origin_code | destination_code | base_fare |
+---------------+-------------+------------------+-----------+
| EK-565        | BLR         | DXB              |     21500 |
| SQ-503        | MAA         | SIN              |     24800 |
| AI-111        | DEL         | LHR              |     52000 |
| EK-565        | BLR         | DXB              |     21000 |
+---------------+-------------+------------------+-----------+
4 rows in set
```

Explanation: IN is shorthand for several OR comparisons on the same column.

Q6 NULL test: cancelled tickets that released their seat

```sql
SELECT ticket_id, booking_id, flight_id, seat_no, status
FROM ticket
WHERE seat_no IS NULL;
```

```
OUTPUT
+-----------+------------+-----------+---------+-----------+
| ticket_id | booking_id | flight_id | seat_no | status    |
+-----------+------------+-----------+---------+-----------+
|        14 |          9 |         6 | NULL    | Cancelled |
|        30 |         20 |        20 | NULL    | Cancelled |
+-----------+------------+-----------+---------+-----------+
2 rows in set
```

Explanation: NULL cannot be compared with =; IS NULL is required. These are the tickets whose seat was released on cancellation.

Q7 DISTINCT: nationalities of registered passengers

```sql
SELECT DISTINCT nationality FROM passenger;
```

```
OUTPUT
+-------------+
| nationality |
+-------------+
| Indian      |
| British     |
| Bhutanese   |
+-------------+
3 rows in set
```

Explanation: DISTINCT removes duplicate values from the result.

3. Sorting Results (ORDER BY)

Q8 Five most expensive flights

```sql
SELECT flight_number, origin_code, destination_code, base_fare
FROM flight
ORDER BY base_fare DESC
LIMIT 5;
```

```
OUTPUT
+---------------+-------------+------------------+-----------+
| flight_number | origin_code | destination_code | base_fare |
+---------------+-------------+------------------+-----------+
| AI-111        | DEL         | LHR              |     52000 |
| SQ-503        | MAA         | SIN              |     24800 |
| EK-567        | DXB         | BLR              |     22300 |
| EK-565        | BLR         | DXB              |     21500 |
| EK-565        | BLR         | DXB              |     21000 |
+---------------+-------------+------------------+-----------+
5 rows in set
```

Explanation: ORDER BY ... DESC sorts from highest to lowest; LIMIT 5 keeps the first five rows. (MySQL/PostgreSQL syntax is identical; SQL Server uses TOP 5.)

Q9 Passengers sorted by nationality, then by surname

```sql
SELECT nationality, last_name, first_name
FROM passenger
ORDER BY nationality ASC, last_name ASC
LIMIT 10;
```

```
OUTPUT
+-------------+-----------+------------+
| nationality | last_name | first_name |
+-------------+-----------+------------+
| Bhutanese   | Dorjee    | Tenzin     |
| British     | Thomas    | Emily      |
| Indian      | Bose      | Ishita     |
| Indian      | Desai     | Pooja      |
| Indian      | Faizan    | Mohammed   |
| Indian      | Gupta     | Sanjay     |
| Indian      | Gupta     | Meera      |
| Indian      | Iyer      | Ananya     |
| Indian      | Khan      | Farah      |
| Indian      | Krishnan  | Divya      |
+-------------+-----------+------------+
10 rows in set
```

Explanation: Several sort keys are applied in order: rows with the same nationality are sorted by last name.

Q10 Flight schedule of 2 November in time order

```sql
SELECT flight_number, origin_code, destination_code, departure_time
FROM flight
WHERE departure_time LIKE '2026-11-02%'
ORDER BY departure_time;
```

```
OUTPUT
+---------------+-------------+------------------+---------------------+
| flight_number | origin_code | destination_code | departure_time      |
+---------------+-------------+------------------+---------------------+
| 6E-501        | BLR         | DEL              | 2026-11-02 06:00:00 |
| AI-807        | BLR         | BOM              | 2026-11-02 07:30:00 |
| 6E-502        | DEL         | BLR              | 2026-11-02 10:00:00 |
| AI-808        | BOM         | BLR              | 2026-11-02 11:00:00 |
+---------------+-------------+------------------+---------------------+
4 rows in set
```

Explanation: WHERE filters first, then ORDER BY sorts what is left. This is a typical departure-board query.

4. Aggregate Functions

Q11 COUNT: number of passengers, bookings and tickets

```sql
SELECT (SELECT COUNT(*) FROM passenger) AS passengers,
       (SELECT COUNT(*) FROM booking)   AS bookings,
       (SELECT COUNT(*) FROM ticket)    AS tickets,
       (SELECT COUNT(DISTINCT passenger_id) FROM ticket) AS travelling_passengers;
```

```
OUTPUT
+------------+----------+---------+----------------------+
| passengers | bookings | tickets | travelling_passengers |
+------------+----------+---------+----------------------+
|         25 |       20 |      30 |                   23 |
+------------+----------+---------+----------------------+
1 row in set
```

Explanation: COUNT(*) counts rows; COUNT(DISTINCT col) counts different values. 25 passengers are registered but only 23 have a ticket.

Q12 SUM, AVG, MIN, MAX of ticket fares (active tickets)

```sql
SELECT COUNT(*)  AS tickets,
       SUM(fare)  AS total_revenue,
       ROUND(AVG(fare), 2) AS average_fare,
       MIN(fare)  AS cheapest,
       MAX(fare)  AS most_expensive
FROM ticket
WHERE status <> 'Cancelled';
```

```
OUTPUT
+---------+--------------+--------------+----------+---------------+
| tickets | total_revenue | average_fare | cheapest | most_expensive |
+---------+--------------+--------------+----------+---------------+
|      28 |        465900 |     16639.29 |     3000 |         145600 |
+---------+--------------+--------------+----------+---------------+
1 row in set
```

Explanation: All five aggregate functions in one query. Cancelled tickets are excluded by WHERE before aggregation.

Q13 Flight base-fare statistics

```sql
SELECT MIN(base_fare) AS min_fare, MAX(base_fare) AS max_fare,
       ROUND(AVG(base_fare), 2) AS avg_fare, SUM(base_fare) AS sum_of_fares
FROM flight
WHERE status <> 'Cancelled';
```

```
OUTPUT
+----------+----------+----------+--------------+
| min_fare | max_fare | avg_fare | sum_of_fares |
+----------+----------+----------+--------------+
|     2500 |    52000 |  9594.74 |       182300 |
+----------+----------+----------+--------------+
1 row in set
```

Explanation: Aggregates summarise a whole table into one row. MIN and MAX also work on dates and text.

Q14 Earliest and latest departure

```sql
SELECT MIN(departure_time) AS first_departure, MAX(departure_time) AS last_departure FROM flight;
```

```
OUTPUT
+---------------------+---------------------+
| first_departure     | last_departure      |
+---------------------+---------------------+
| 2026-10-20 06:00:00 | 2026-11-09 19:00:00 |
+---------------------+---------------------+
1 row in set
```

Explanation: MIN and MAX on date-time text values give the time span of the schedule.

5. Grouping Results (GROUP BY)

Q15 Tickets and average fare per travel class

```sql
SELECT travel_class, COUNT(*) AS tickets, ROUND(AVG(fare), 2) AS avg_fare, SUM(fare) AS revenue
FROM ticket
WHERE status <> 'Cancelled'
GROUP BY travel_class
ORDER BY revenue DESC;
```

```
OUTPUT
+-----------------+---------+----------+---------+
| travel_class    | tickets | avg_fare | revenue |
+-----------------+---------+----------+---------+
| Business        |       4 |    67495 |  269980 |
| Economy         |      21 |  8378.10 |  175940 |
| Premium Economy |       3 |     6660 |   19980 |
+-----------------+---------+----------+---------+
3 rows in set
```

Explanation: GROUP BY forms one group per travel class and the aggregates are computed per group. Business class has few tickets but a large share of revenue.

Q16 Number of flights and average base fare per origin airport

```sql
SELECT origin_code, COUNT(*) AS flights, ROUND(AVG(base_fare), 2) AS avg_base_fare
FROM flight
GROUP BY origin_code
ORDER BY flights DESC, origin_code;
```

```
OUTPUT
+-------------+---------+---------------+
| origin_code | flights | avg_base_fare |
+-------------+---------+---------------+
| BLR         |       9 |       7733.33 |
| DEL         |       5 |         14200 |
| BOM         |       3 |       3833.33 |
| DXB         |       1 |         22300 |
| HYD         |       1 |          4100 |
| MAA         |       1 |         24800 |
+-------------+---------+---------------+
6 rows in set
```

Explanation: Every non-aggregated column in SELECT must appear in GROUP BY. Bengaluru is the busiest origin.

Q17 Bookings and value per booking status

```sql
SELECT status, COUNT(*) AS bookings, SUM(total_amount) AS total_value
FROM booking
GROUP BY status;
```

```
OUTPUT
+-----------+----------+-------------+
| status    | bookings | total_value |
+-----------+----------+-------------+
| Cancelled |        2 |       24080 |
| Confirmed |       17 |      462300 |
| Pending   |        1 |        3600 |
+-----------+----------+-------------+
3 rows in set
```

Explanation: Groups bookings by status to summarise confirmed, pending and cancelled business.

Q18 Successful payments by method

```sql
SELECT method, COUNT(*) AS payments, SUM(amount) AS amount_received
FROM payment
WHERE status = 'Success'
GROUP BY method
ORDER BY amount_received DESC;
```

```
OUTPUT
+-------------+----------+-----------------+
| method      | payments | amount_received |
+-------------+----------+-----------------+
| Wallet      |        6 |          274900 |
| Debit Card  |        6 |          144960 |
| UPI         |        3 |           35100 |
| Net Banking |        2 |           19540 |
| Credit Card |        2 |           11880 |
+-------------+----------+-----------------+
5 rows in set
```

Explanation: WHERE removes failed and refunded rows before grouping, so only money actually received is totalled.

6. Filtering Groups (HAVING)

WHERE filters individual rows *before* grouping; **HAVING** filters whole groups *after* aggregation, so it can use aggregate functions.

Q19 Origin airports with more than two flights

```sql
SELECT origin_code, COUNT(*) AS flights
FROM flight
GROUP BY origin_code
HAVING COUNT(*) > 2;
```

```
OUTPUT
+-------------+---------+
| origin_code | flights |
+-------------+---------+
| BLR         |       9 |
| BOM         |       3 |
| DEL         |       5 |
+-------------+---------+
3 rows in set
```

Explanation: Only groups whose flight count exceeds 2 are kept.

Q20 Bookings that contain at least two active tickets

```sql
SELECT booking_id, COUNT(*) AS tickets, SUM(fare) AS amount
FROM ticket
WHERE status <> 'Cancelled'
GROUP BY booking_id
HAVING COUNT(*) >= 2
ORDER BY amount DESC;
```

```
OUTPUT
+------------+---------+--------+
| booking_id | tickets | amount |
+------------+---------+--------+
|         12 |       2 | 108360 |
|         17 |       2 |  53520 |
|         13 |       2 |  49600 |
|          5 |       3 |  15300 |
|          3 |       2 |   9360 |
|          2 |       2 |   9020 |
|         16 |       2 |   8060 |
|          8 |       2 |   7800 |
|         10 |       2 |   6000 |
+------------+---------+--------+
9 rows in set
```

Explanation: Group bookings (family or group travel) are found with HAVING COUNT(*) >= 2.

Q21 Payment methods that brought in more than 20,000

```sql
SELECT method, SUM(amount) AS received
FROM payment
WHERE status = 'Success'
GROUP BY method
HAVING SUM(amount) > 20000
ORDER BY received DESC;
```

```
OUTPUT
+------------+----------+
| method     | received |
+------------+----------+
| Wallet     |   274900 |
| Debit Card |   144960 |
| UPI        |    35100 |
+------------+----------+
3 rows in set
```

Explanation: WHERE keeps successful payments; HAVING then discards methods with a small total.

Q22 Passengers who have spent more than 10,000 in total

```sql
SELECT passenger_id, COUNT(*) AS tickets, SUM(fare) AS total_spent
FROM ticket
WHERE status <> 'Cancelled'
GROUP BY passenger_id
HAVING SUM(fare) > 10000
ORDER BY total_spent DESC;
```

```
+--------------+---------+-------------+
| passenger_id | tickets | total_spent |
+--------------+---------+-------------+
|           21 |       1 |      145600 |
|           17 |       1 |       54180 |
|           18 |       1 |       54180 |
|            3 |       3 |       35950 |
|            2 |       2 |       31270 |
|           19 |       1 |       24800 |
|           20 |       1 |       24800 |
|            5 |       1 |       16020 |
|            1 |       3 |       13060 |
+--------------+---------+-------------+
9 rows in set
```

Explanation: A simple customer-value report: high-spending travellers (candidates for a loyalty programme).

7. Manipulating Data (INSERT, UPDATE, DELETE)

The following data-manipulation statements are run in a transaction and rolled back at the end, so the sample data stays unchanged for the next assignment.

```sql
BEGIN;
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Nisha', 'Rao', 'nisha.rao@example.com', '+91-9811122233', '1994-05-05', 'F', 'N7654321');
UPDATE flight SET status = 'Delayed', arrival_time = '2026-11-03 11:55:00'
WHERE flight_number = 'SG-612';
DELETE FROM passenger WHERE passenger_id = 26;
SELECT flight_number, status, arrival_time FROM flight WHERE flight_number = 'SG-612';
ROLLBACK;
```

```
Query OK, 1 row affected  (INSERT)
Query OK, 1 row affected  (UPDATE)
Query OK, 1 row affected  (DELETE)
+---------------+---------+---------------------+
| flight_number | status  | arrival_time        |
+---------------+---------+---------------------+
| SG-612        | Delayed | 2026-11-03 11:55:00 |
+---------------+---------+---------------------+
1 row in set
```

Explanation: INSERT adds a new passenger, UPDATE changes the status and arrival time of flight SG-612 (a delay), and DELETE removes the test passenger again. The final SELECT shows the delayed flight inside the transaction; ROLLBACK then restores the original data.

8. Conclusion

The queries demonstrate filtering with WHERE (comparison, BETWEEN, LIKE, IN, IS NULL), sorting with ORDER BY, the aggregate functions COUNT, SUM, AVG, MIN and MAX, grouping with GROUP BY, and group filtering with HAVING, as well as INSERT, UPDATE and DELETE. They answer practical questions of the airline: which flights depart from a city, which classes earn the most, how customers pay, and who the high-value passengers are.